



微信搜一搜

Q 爪哇繆斯

反射&泛型&集合



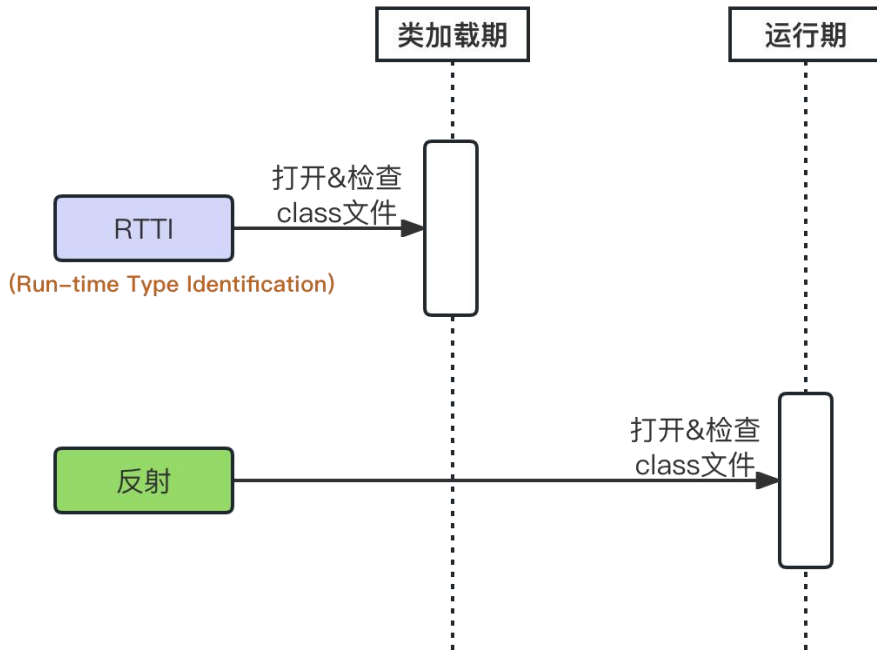
反 射

类加载区别

- 当我们刚接触java语言的时候，我们最常写的代码应该就是初始化某个对象，然后调用该方法。例如：

```
Student student = new Student();  
student.say();
```

- 上面的这种用法的前提是，我们已经知道要去操作Student这个具体的实例对象了。
- 当我们想在运行期才能指定具体对象的类型或调用的某个方法、属性时，普通的编码方式将无法实现需求；反射将开始自己的表演了。反射为什么可以实现呢，这个就要先介绍一下new对象和反射机制在类加载的过程中有什么区别了。请见右图所示：



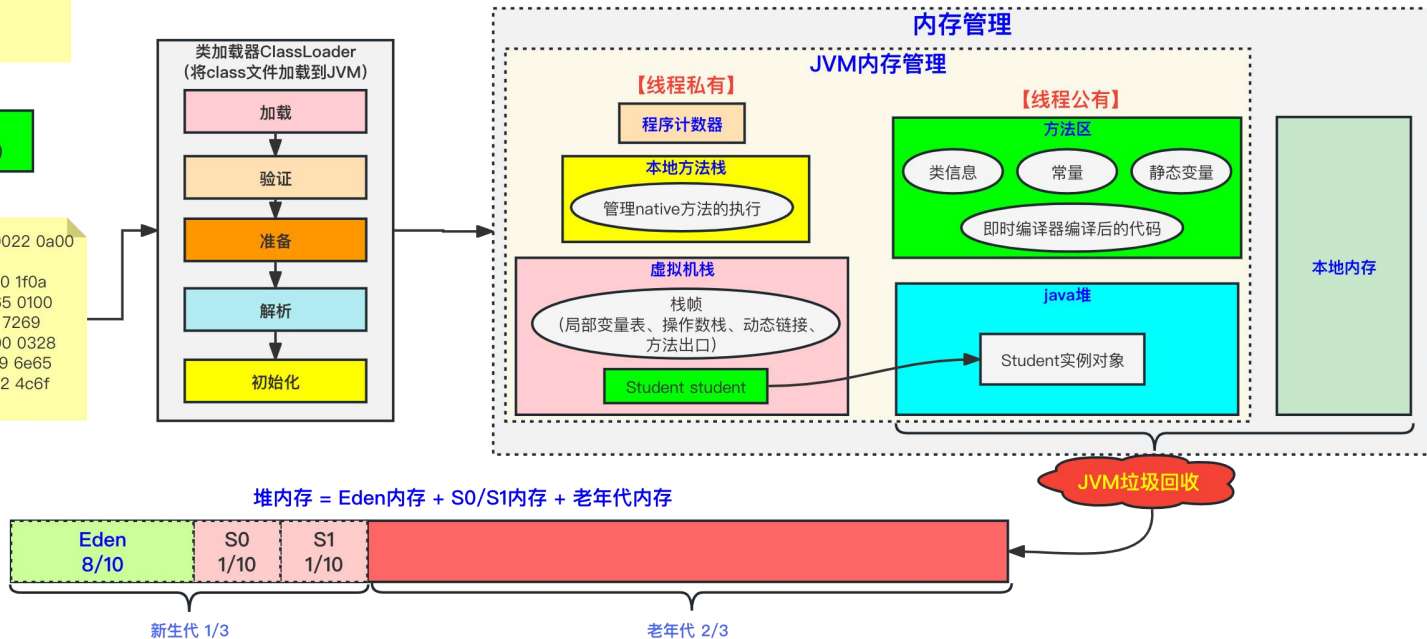
JVM类加载流程和内存结构

```
【Java源文件】 public class Student {  
    private String name;  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return this.name;  
    }  
    public static void main(String[] args) {  
        Student student1 = new Student();  
        student1.setName("muse");  
    }  
}
```

类编译器
(将java源码编译为class文件)

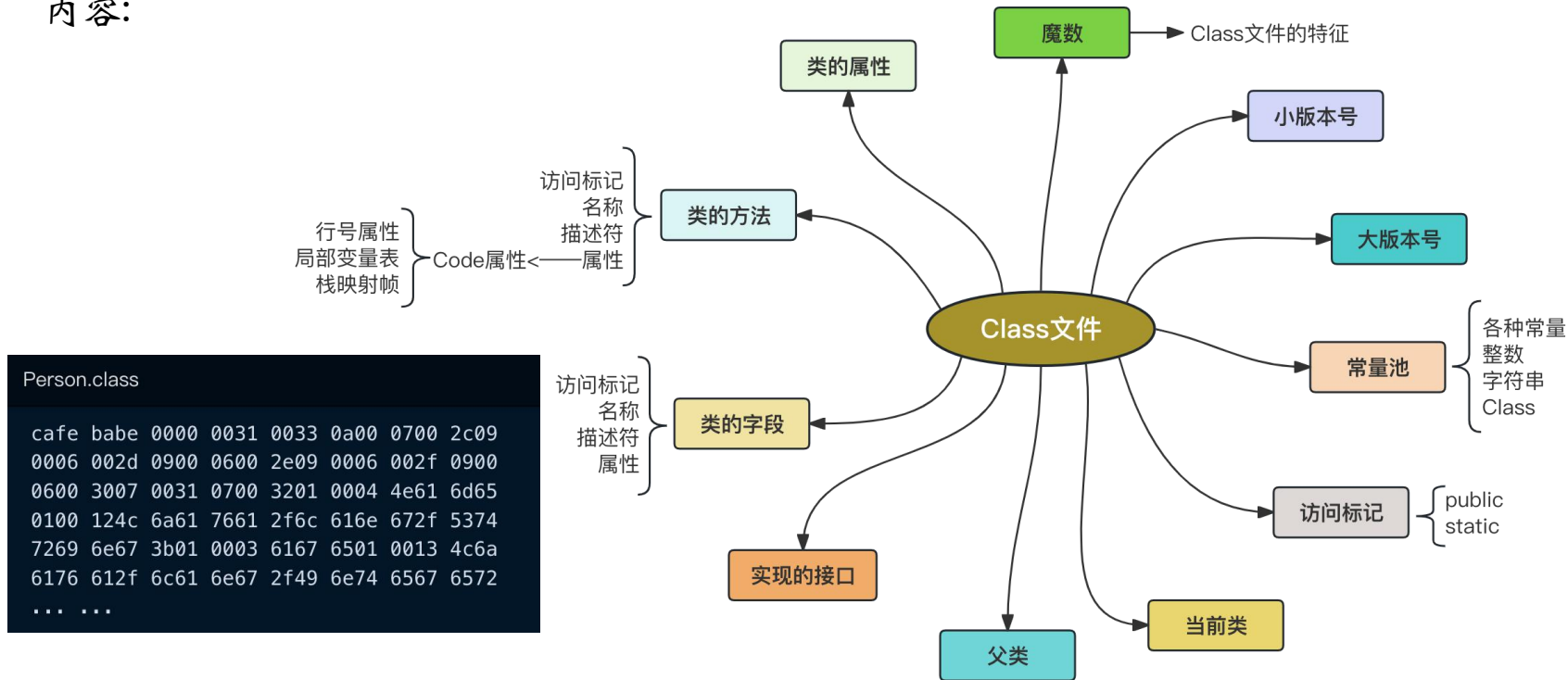
```
【Class文件】 cafe babe 0000 0031 0022 0a00  
0700 1c09  
0003 001d 0700 1e0a 0003 001c 0800 1f0a  
0003 0020 0700 2101 0004 6e61 6d65 0100  
124c 6a61 7661 2f6c 616e 672f 5374 7269  
6e67 3b01 0006 3c69 6e69 743e 0100 0328  
2956 0100 0443 6f64 6501 000f 4c69 6e65  
4e75 6d62 6572 5461 626c 6501 0012 4c6f  
... ..
```

➤ 既然反射与类和对象息息相关，那么我们就先从Java虚拟机的视角，从整体上线了解一下类的加载过程。流程图如下所示：



Class文件包含的内容

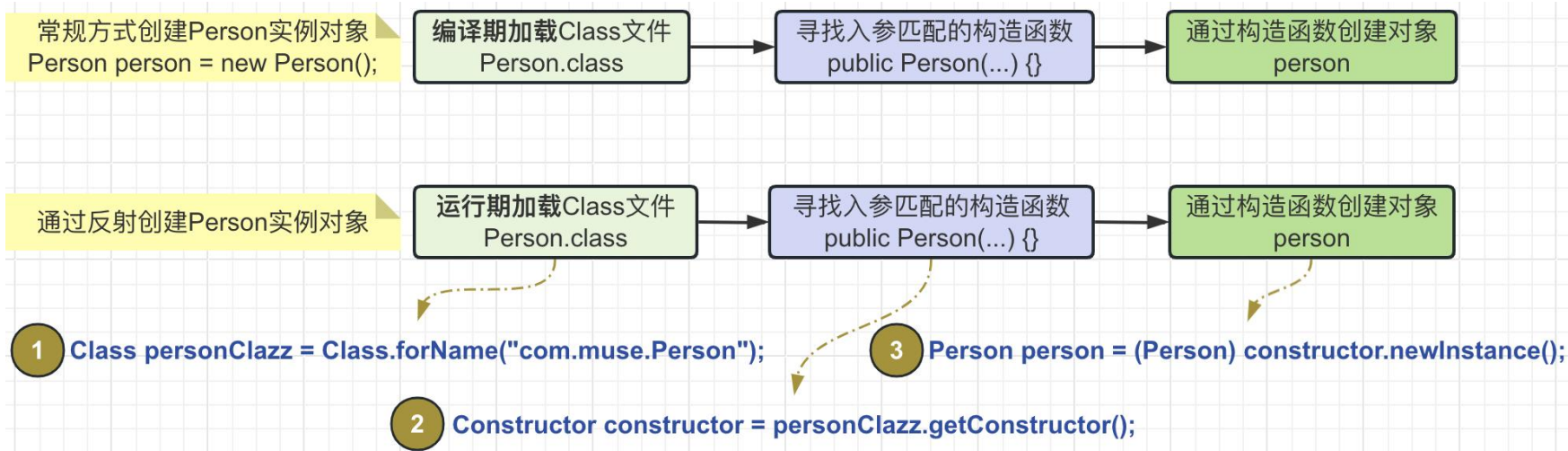
- Class文件中，几乎包含了所有类相关的信息，如：**类名**、**包名**、**属性**、**方法**等等。那么，Class对象其实就是我们获得的某一个类型字节码的引用对象。Class文件中包含如下主要内容：



创建实例对象的步骤对比

➤ 创建对象的过程，通过下图我们能够看出来，获得对象实例无论是使用new对象的方式，还是使用反射的方式，大致都是需要以下3步的，即：

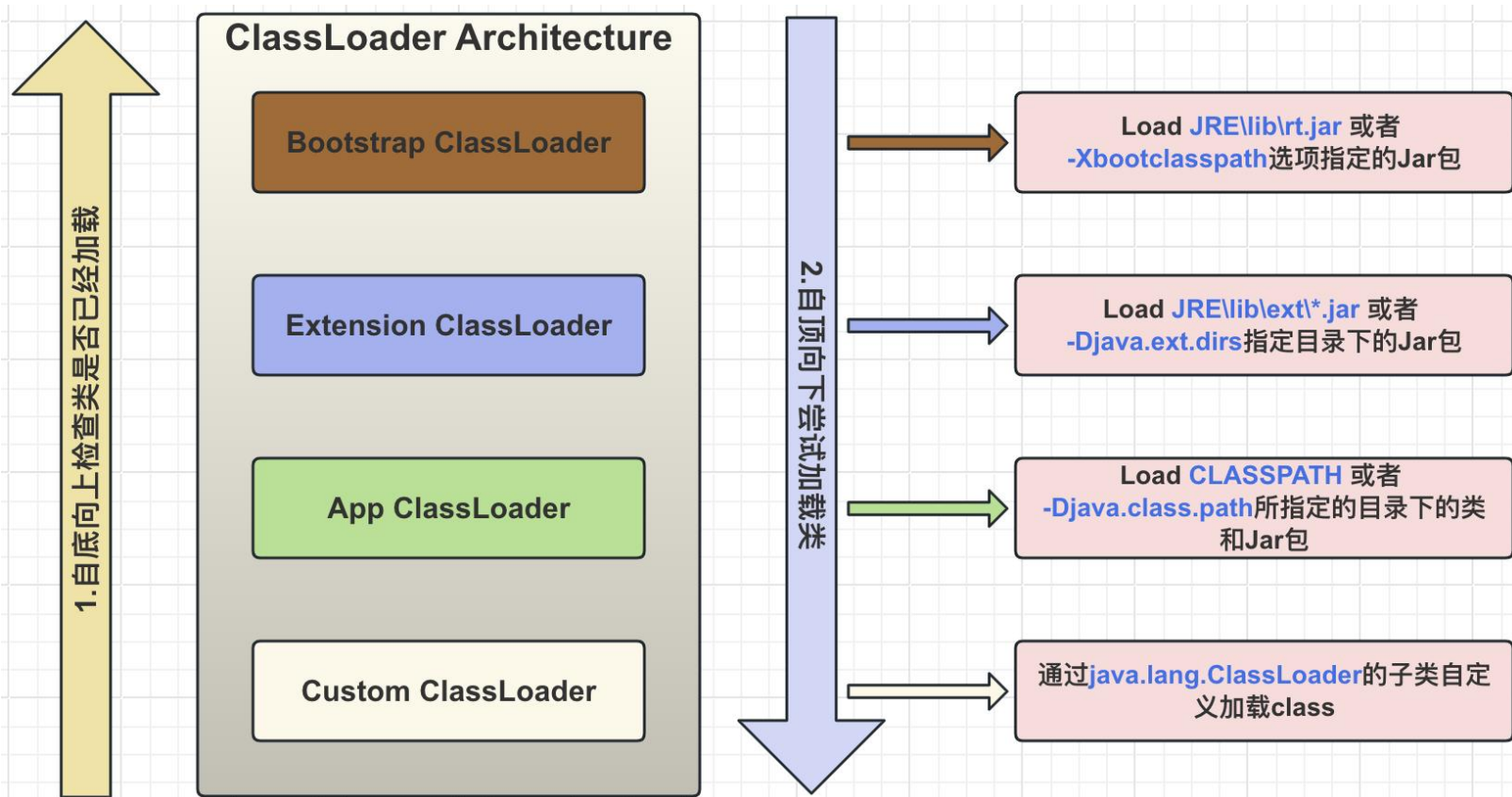
- ① 加载Class文件 (*获得Class实例的方式有几种?*)；
- ② 查找入参匹配的构造函数；
- ③ 通过构造函数创建实例对象；



反射相关的操作演示

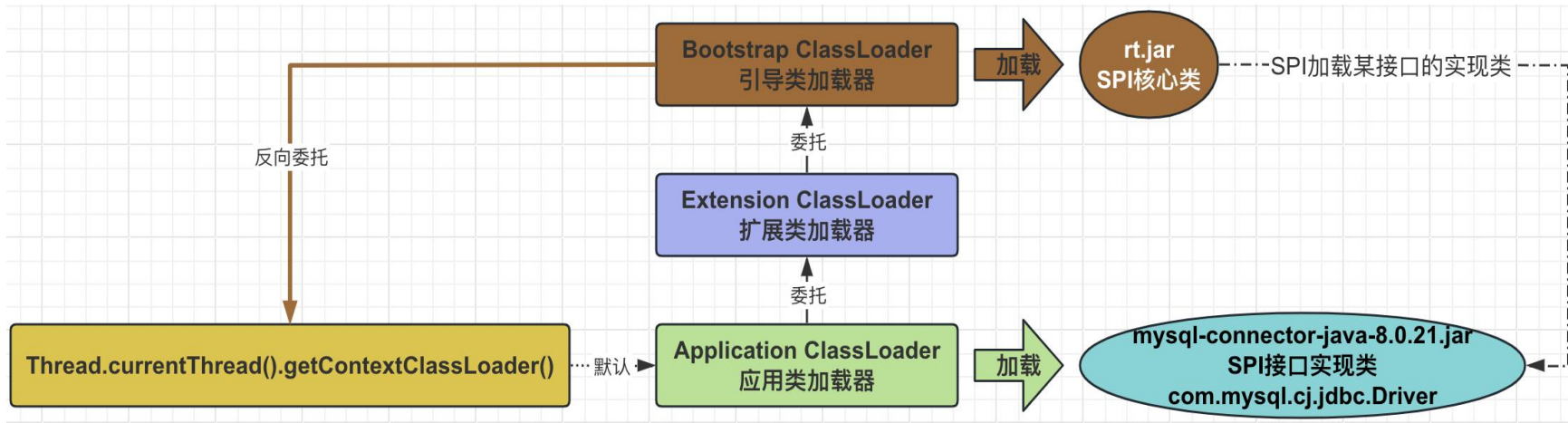
- 演示1: 通过无参构造函数创建实例对象。
- 演示2: 通过有参构造函数创建实例对象。
- 演示3: 反射通过私有构造函数创建对象，破坏单例模式。
- 演示4: 通过反射获得类的public属性值，演示getField与getDeclaredField两者的区别。
- 演示5: 通过反射获得类的private、protected、默认访问修饰符的属性值。
- 演示6: 通过反射获得类的private方法。
- 演示7: 我们采用反射机制来实现一个工具BeanUtils，可以将一个对象属性相同的值赋值给另一个对象。
- 演示8: 通过mapstruct实现两个实体间的属性值复制。

双亲委派机制



为什么需要SPI破坏双亲委派机制？

- SPI是Service Provider Interface的缩写，它是Java提供的一套用于被第三方开发或者实现的API接口，可以用于模块化、解耦，插件机制等场景。
- SPI具体的使用方式是：首先定义一个接口，然后在项目的src/main/resources/META-INF/services目录下创建一个同名的文件，文件内容是实现该接口的具体实现类。然后在程序运行时，JVM会查找并加载这些实现类。





泛型

什么是泛型

- 早期Java是使用Object来代表任意类型的，但是向下转型有强转的问题，这样程序并不安全。
- 针对List、Set、Map等集合类型，它们对存储的元素类型是没有任何限制的。假如向List中存储Dog类型的对象，但是有人把Cat对象也存储到这个List中了，那么在编译上是没有任何语法错误的。
- 所有使用该泛型参数的地方都被统一化，保证类型一致。如果未指定具体类型，默认是Object类型。集合体系中的所有类都增加了泛型，**泛型也主要用在集合**。
- 下面我们就针对泛型类、泛型方法、泛型类派生出的子类、类型通配符、泛型擦除、桥接方法等内容进行一一介绍。

泛型类

- 泛型类就是把泛型定义在类上，用户使用该类的时候，才把类型明确下来。这样的话，用户明确了什么类型，该类就代表着什么类型，用户在使用的时候就不用担心强转的问题，和运行时转换异常的问题了，如下所示：

```
public class ClassGenericity {
    public static void main(String[] args) {
        ObjectTool<String> stringTool = new ObjectTool<>(); // 指定元素类型为String
        stringTool.setObj("muse");
        System.out.println(stringTool.getObj());

        ObjectTool<Integer> integerTool = new ObjectTool<>(); // 指定元素类型为Integer
        // integerTool.setObj("muse"); // 编译报错
        integerTool.setObj(10);
        System.out.println(integerTool.getObj());
    }
    /** 构建可以存储任何类型对象的工具类 */
    static class ObjectTool<T> {
        private T obj;
        public T getObj() {return obj;}
        public void setObj(T obj) {this.obj = obj;}
    }
}
```

泛型方法

- 除了在类上使用泛型，我们可能就仅仅在某个方法上需要使用泛型，外界仅仅是关心该方法，不关心类其他的属性，这样的话，我们在整个类上定义泛型，未免就有些大题小作了。那么此时，我们可以采用泛型方法。

```
public class MethodGenericity<T> {  
    public static void main(String[] args) {  
        ObjectTool tool = new ObjectTool(); //创建对象  
        //调用方法,传入的参数是什么类型,T就是什么类型  
        tool.show("hello");  
        tool.show(12);  
        tool.show(12.5f);  
    }  
  
    static class ObjectTool {  
        public <T> void show(T t) { //定义泛型方法..  
            System.out.println(t);  
        }  
    }  
}
```

泛型类派生出的子类

➤ 前面我们已经定义了泛型类，泛型类是拥有泛型这个特性的类，它本质上还是一个Java类，那么它就可以被继承或实现。这里分两种情况：

① 子类明确泛型类的类型参数变量

```
class InterImpl1 implements Inter<String> {  
    @Override  
    public void show(String s) {  
        System.out.println(s);  
    }  
}
```

② 子类不明确泛型类的类型参数变量

```
class InterImpl2<T> implements Inter<T> {  
    @Override  
    public void show(T t) {  
        System.out.println(t);  
    }  
}
```

➤ 我们创建一个Inter接口，在该接口上使用泛型，代码如下所示：

```
interface Inter<T> {  
    void show(T t);  
}
```

类型通配符

- `List<?>` 表示元素类型未知的`List`，它可以匹配任何类型的元素。声明 `List<?> list` 后，不能向集合中添加元素，因为无法确定集合的元素类型，唯一例外的是 `null`。

```
public class TypeWildcard1 {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("aaa");
        list.add("bbb");
        print1(list);

        List<Integer> list2 = new ArrayList<>();
        list2.add(111);
        list2.add(222);
        print2(list2);
    }

    public static void print1(List<String> list) {
        for (String str : list) System.out.println(str);
    }

    public static void print2(List<Integer> list) {
        for (Integer i : list) System.out.println(i);
    }
}
```

```
public class TypeWildcard2 {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("aaa");
        list.add("bbb");
        print(list);

        List<Integer> list2 = new ArrayList<>();
        list2.add(111);
        list2.add(222);
        print(list2);
    }

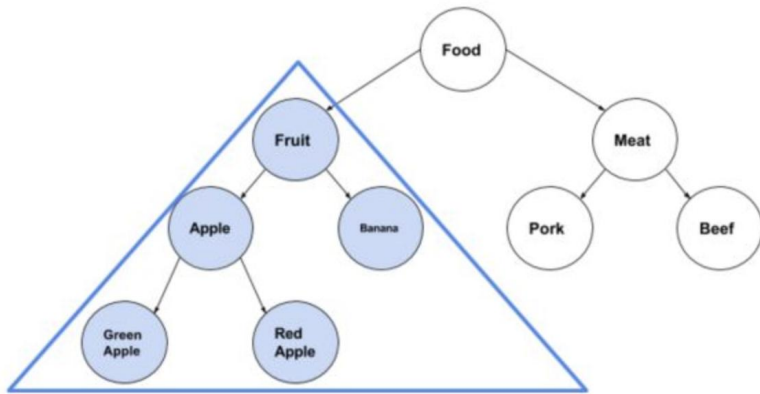
    public static void print(List<?> list) {
        // 因为无法确定集合的元素类型，所以不能向集合中添加元素，会报编译错误
        list.add(list.get(0));
        list.add(null); // 唯一例外的是可以添加null
        for (Object obj : list) System.out.println(obj);
    }
}
```

泛型的上限和下限

泛型的上限：

- 格式：类型名称 `<? extends 类>` 对象名称
- 意义：只能接收该类型及其子类

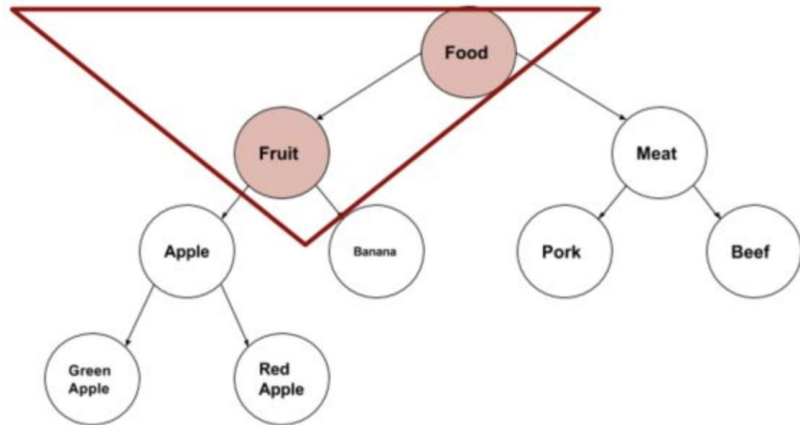
`Plate<? extends Fruit>`



泛型的下限：

- 格式：类型名称 `<? super 类>` 对象名称
- 意义：只能接收该类型及其父类型

`Plate<? super Fruit>`



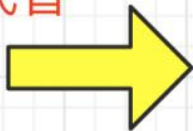
类型擦除

- 泛型是提供给javac编译器使用的，它可以作为类型的限制，让编译器在源代码级别上，挡住非法类型的数据。
- 但是在JDK1.5之前没有泛型的概念，为了能够与之前版本代码兼容，编译器编译完带有泛型的java程序后，生成的class字节码文件中将不再带有泛型信息，这个过程称之为“擦除”。

```
public interface Animal<T> {  
    void eat(T t);  
}
```

擦除掉泛型，以 Object 代替

```
public class Cat implements Animal<String> {  
    @Override  
    public void eat(String s) {  
        System.out.println("cat eat " + s);  
    }  
}
```



```
public interface Animal {  
    void eat(Object t);  
}
```

```
public class Cat implements Animal {  
    @Override  
    public void eat(String s) {  
        System.out.println("cat eat " + s);  
    }  
}
```

桥接方法的作用是什么

- 桥接方法是jdk1.5引入泛型后，为使java泛型方法生成的字节码与jdk1.5版本之前的字节码兼容由编译器自动生成的。可用 `method.isBridge()` 判断method是否是桥接方法。

```
public interface Animal {  
    void eat(Object t);  
}
```

```
public class Cat implements Animal {  
    @Override  
    public void eat(String s) {  
        System.out.println("cat eat " + s);  
    }  
}
```

桥接方法

```
public interface Animal {  
    void eat(Object t);  
}
```

```
public class Cat implements Animal {  
    @Override 自动生成的桥接方法
```

```
    public void eat(Object s) {  
        this.eat((String) s);  
    }
```

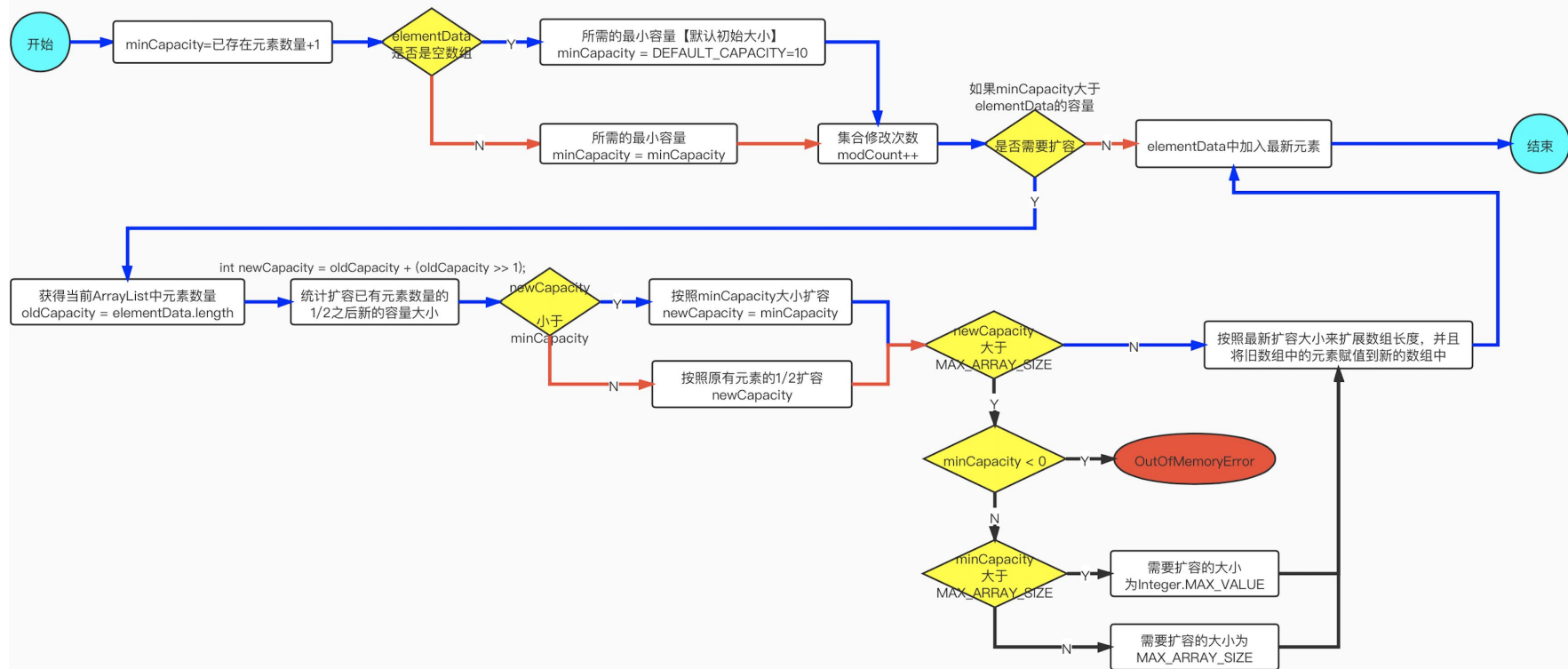
```
    public void eat(String s) {  
        System.out.println("cat eat " + s);  
    }  
}
```



集合

ArrayList源码解析

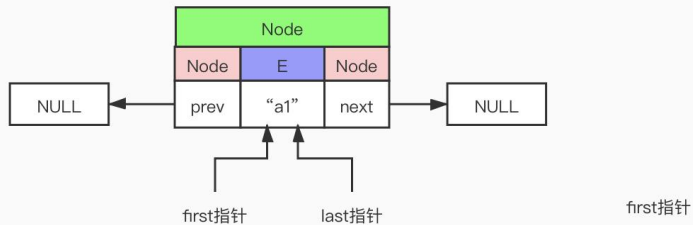
```
arrayList.add("a1");
```



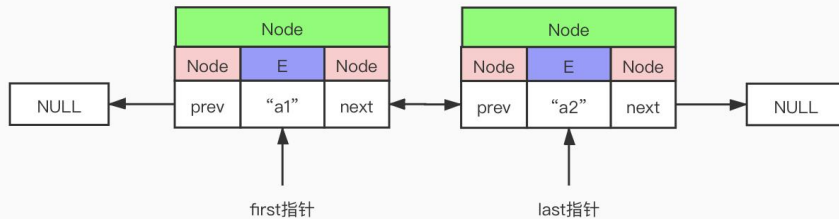
LinkedList源码解析

```
LinkedList linkedList = new LinkedList();
```

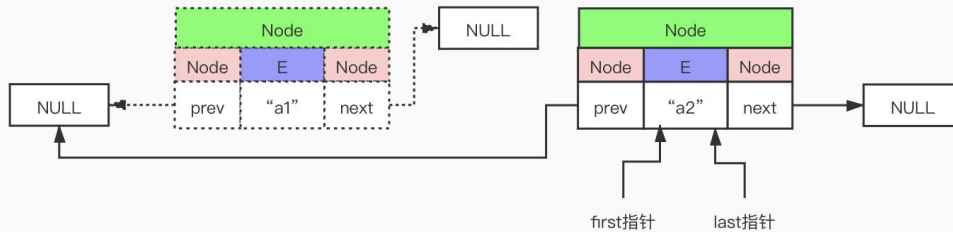
```
linkedList.add("a1");
```



```
linkedList.add("a2");
```

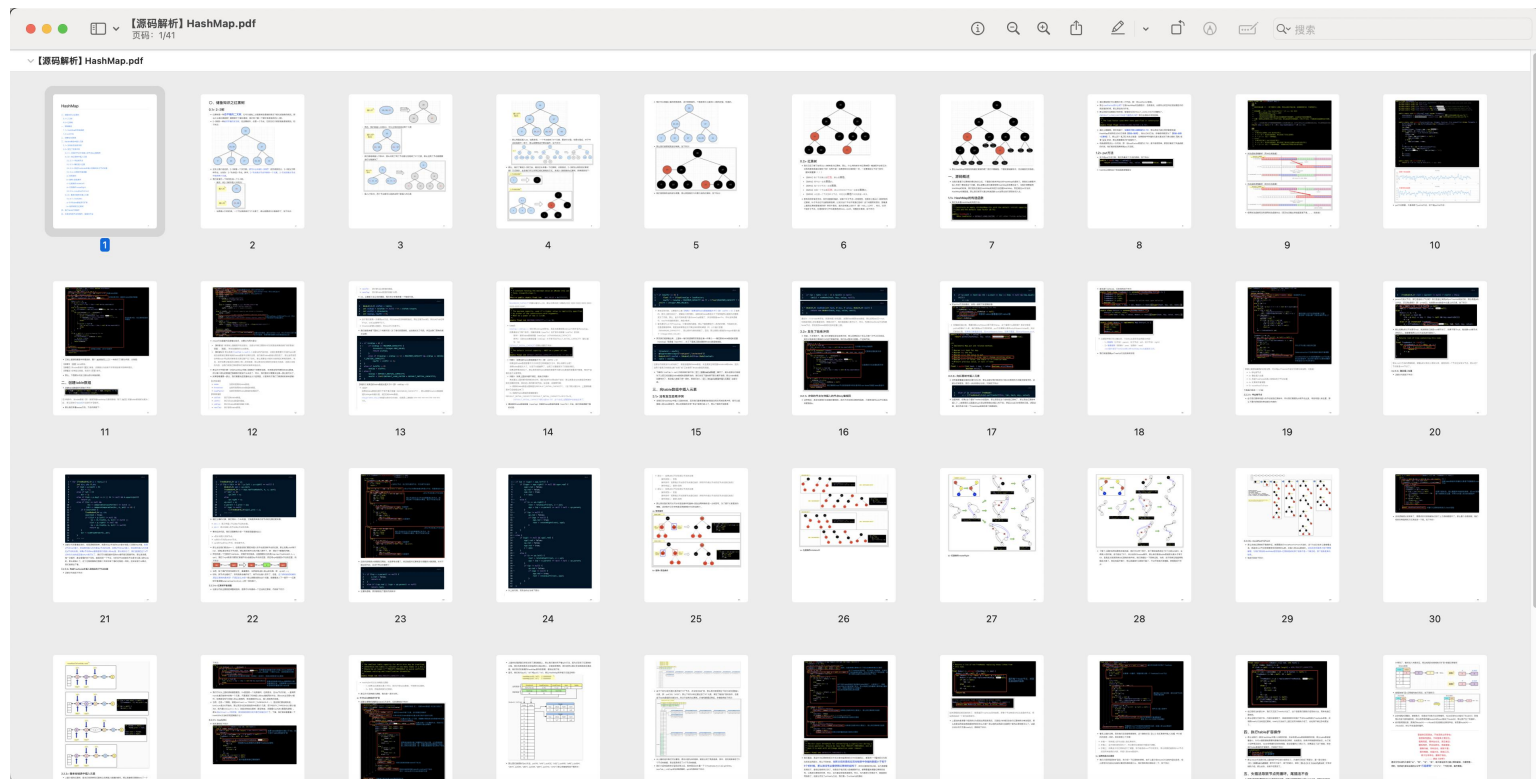


```
linkedList.remove(1);
```



HashMap源码解析

➤ 详情请见《源码解析——HashMap》文章



End



微信搜一搜



爪哇繆斯

演示代码: https://gitee.com/muse_1/java-demo.git
JDK源码: https://gitee.com/muse_1/jdk1.8.0_261.git